

Core Java

Control flow — if/else, switch, switch expressions (Java 14+)
Explain with examples and with most commonly asked interview questions

Here's a **simple + interview-focused explanation** of Java control flow:

♦ 1. if / else (Decision Making)

Used when you want to execute code based on a condition.

✓ Basic Syntax

```
Java
if (condition) {
    // runs if condition is true
} else {
    // runs if condition is false
}
```

✓ Example

```
Java
int age = 18;

if (age >= 18) {
    System.out.println("Eligible to vote");
} else {
    System.out.println("Not eligible");
}
```

✓ Multiple Conditions (else if)

```
Java
int marks = 75;

if (marks >= 90) {
    System.out.println("A Grade");
} else if (marks >= 60) {
    System.out.println("B Grade");
} else {
    System.out.println("Fail");
}
```

♦ 2. switch Statement (Traditional)

Used when you have multiple fixed values.

✓ Syntax

```
Java
```

```
switch (value) {
  case 1:
    // code
    break;
  case 2:
    // code
    break;
  default:
    // code
}
```

✓ Example

```
Java
int day = 2;

switch (day) {
  case 1:
    System.out.println("Monday");
    break;
  case 2:
    System.out.println("Tuesday");
    break;
  default:
    System.out.println("Invalid day");
}
```

⚠ Important Concept: fall-through

If you don't use **break**, execution continues.

```
Java
int x = 1;

switch (x) {
  case 1:
    System.out.println("One");
  case 2:
    System.out.println("Two");
}
```

👉 Output:

```
One
Two
```

◆ 3. switch Expression (Java 14+ 🔥)

Modern and cleaner version of switch.

✓ Key Features

- No need for **break**
- Returns a value
- Uses ->

✓ Example 1 (Simple)

```
Java
int day = 2;

String result = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    default -> "Invalid";
};

System.out.println(result);
```

✓ Example 2 (Multiple values)

```
Java
int day = 6;

String type = switch (day) {
    case 1, 2, 3, 4, 5 -> "Weekday";
    case 6, 7 -> "Weekend";
    default -> "Invalid";
};
```

✓ Example 3 (Using yield)

```
Java
int num = 5;

String result = switch (num) {
    case 1 -> "One";
    case 2 -> "Two";
    default -> {
        yield "Other number";
    }
};
```

🔥 Difference: switch vs switch expression

Feature	switch (old)	switch expression
break needed	Yes	No
returns value	No	Yes
syntax	verbose	clean
fall-through	Yes	No

🎯 Most Asked Interview Questions

? 1. Difference between if-else and switch?

👉 Answer:

- **if-else** → used for **range/complex conditions**

- `switch` → used for **fixed values** (`int`, `String`, `enum`)

? 2. Can we use String in switch?

👉 Yes (from Java 7)

```
Java
String day = "MON";

switch (day) {
    case "MON":
        System.out.println("Monday");
}
```

? 3. Which is faster: if-else or switch?

👉 Depends:

- `switch` is faster when many fixed cases (uses jump table internally)
- `if-else` better for complex conditions

? 4. What happens if break is missing?

👉 Fall-through happens (next cases execute)

? 5. Can switch work with boolean?

👉 ❌ No

Supported types:

- `int`, `char`, `byte`, `short`
- `String`
- `enum`

? 6. Why switch expression introduced?

👉 To:

- Reduce boilerplate
- Avoid bugs (missing `break`)
- Make code cleaner & functional

? 7. Can we return value from switch?

👉 Yes (Java 14+ switch expression)

? 8. When should you NOT use switch?

👉 Avoid when:

- Conditions are ranges (>, <)
- Complex logic required

💡 Pro Tip (Interview Gold)

👉 Always say this:

“Use **if-else** for dynamic conditions and **switch** for fixed values.
Prefer **switch expression (Java 14+)** for cleaner and safer code.”

If you want, I can give you:

- ✓ Real interview coding questions
- ✓ Tricky MCQs (asked in TCS/Accenture)
- ✓ Mini test to practice

Just tell me 👍

Loops — for, while, do-while, enhanced for-each

Here's a **clear + interview-focused breakdown of Java loops** 👉

🔄 1. for Loop (Most Common)

Used when you know **how many times** you want to run something.

✓ Syntax

```
Java
for (initialization; condition; update) {
    // code
}
```

✓ Example

Java

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

👉 Output: 1 2 3 4 5

💡 When to use?

- Fixed number of iterations
- Array traversal (index-based)

🔄 2. while Loop

Used when you don't know exact iterations, only a **condition**.

✅ Syntax

```
Java
while (condition) {
    // code
}
```

✅ Example

```
Java
int i = 1;

while (i <= 5) {
    System.out.println(i);
    i++;
}
```

💡 When to use?

- Unknown number of iterations
- Condition-based loops (user input, etc.)

🔄 3. do-while Loop

Runs **at least once**, even if condition is false.

✅ Syntax

```
Java
do {
    // code
} while (condition);
```

✅ Example

```
Java
int i = 10;

do {
    System.out.println(i);
    i++;
} while (i <= 5);
```

👉 Output: 10 (runs once!)

💡 Key Point

👉 Condition checked **after** execution

🔄 4. Enhanced for-each Loop (Java 5+)

Used to iterate over **arrays or collections**

✅ Syntax

```
Java
for (datatype var : collection) {
    // code
}
```

✅ Example

```
Java
int[] arr = {10, 20, 30};

for (int num : arr) {
    System.out.println(num);
}
```

👉 Output:

```
10
20
30
```

💡 When to use?

- When you don't need index
- Cleaner and less error-prone

🔥 Difference Summary

Loop	Use Case	Condition Check	Runs Minimum
for	Known count	Before	0 times
while	Unknown count	Before	0 times

Loop	Use Case	Condition Check	Runs Minimum
do-while	Must run once	After	1 time
for-each	Arrays/collections	Before	0 times

Most Asked Interview Questions

? 1. Difference between for and while?

👉 Answer:

- **for** → when iterations are known
- **while** → when condition-based

? 2. Difference between while and do-while?

👉 Answer:

- **while** → checks first
- **do-while** → runs at least once

? 3. Can we use break and continue?

✅ **break** → exit loop

```
Java
for (int i = 1; i <= 5; i++) {
    if (i == 3) break;
    System.out.println(i);
}
```

👉 Output: 1 2

✅ **continue** → skip iteration

```
Java
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    System.out.println(i);
}
```

👉 Output: 1 2 4 5

? 4. Infinite loop example?

```
Java
while (true) {
    System.out.println("Running...");
}
```

? 5. Can we use for-each to modify array?

👉 ❌ No (common trap)

```
Java
int[] arr = {1, 2, 3};

for (int x : arr) {
    x = x * 2; // won't change original array
}
```

? 6. Which loop is fastest?

👉 No big difference (compiler optimizes)

👉 Use based on readability, not speed

? 7. Can we skip initialization/update in for loop?

👉 Yes (rare but possible)

```
Java
int i = 1;

for (; i <= 5;) {
    System.out.println(i);
    i++;
}
```

💡 Pro Interview Tip

Say this confidently:

“Use **for loop** when iteration count is known,
while loop when condition-based,
do-while when at least one execution is required,
and **for-each** for clean iteration over collections.”

If you want next level prep, I can give:

✅ Pattern questions (very important 🔥)

- ✓ Tricky output-based questions
- ✓ Interview coding problems (TCS / Accenture level)

Just tell me 👍

break, continue, labeled statements

Here's a **clear + interview-ready explanation** of break, continue, and **labeled statements in Java** 👉

◆ 1. break Statement

👉 Used to **exit a loop or switch immediately**

✓ Example (Loop)

```
Java
for (int i = 1; i <= 5; i++) {
    if (i == 3) break;
    System.out.println(i);
}
```

👉 Output:

```
1
2
```

✓ Example (Switch)

```
Java
int day = 2;

switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
}
```

💡 Key Point

- Stops execution completely from loop/switch

◆ 2. continue Statement

👉 Used to **skip current iteration and move to next**

✓ Example

Java

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    System.out.println(i);
}
```

👉 Output:

```
1
2
4
5
```

💡 Key Point

- Does NOT exit loop
- Only skips current iteration

🔥 Difference: break vs continue

Feature	break	continue
Stops loop	✅ Yes	❌ No
Skips iteration	❌ No	✅ Yes
Works in switch	✅ Yes	❌ No

💎 3. Labeled Statements (🔥 Important for Interviews)

👉 Used with **nested loops** to control outer loop

✅ Syntax

Java

```
labelName:
for (...) {
    for (...) {
        break labelName;
    }
}
```

🔥 Labeled break Example

Java

```
outer:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (i == 2 && j == 2) {
            break outer;
        }
        System.out.println(i + " " + j);
    }
}
```

```
}  
}
```

👉 Output:

```
1 1  
1 2  
1 3  
2 1
```

👉 Stops **entire outer loop**

🔥 Labeled continue Example

```
Java  
outer:  
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {  
        if (j == 2) {  
            continue outer;  
        }  
        System.out.println(i + " " + j);  
    }  
}
```

👉 Output:

```
1 1  
2 1  
3 1
```

👉 Skips to next iteration of **outer loop**

🎯 Most Asked Interview Questions

? 1. Difference between break and continue?

👉 Answer:

- **break** → exits loop completely
- **continue** → skips current iteration

? 2. Can break be used outside loop?

👉 ❌ No (only in loop or switch)

? 3. Can continue be used in switch?

👉 ❌ No (only loops)

? 4. What are labeled statements?

👉 Used to:

- Control outer loops in nested structure
- Avoid complex flags/conditions

? 5. Is labeled break recommended?

👉 ⚠️ Use carefully

- Good for interviews
- Rare in production (can reduce readability)

? 6. What happens in nested loops without label?

👉 Only inner loop breaks

```
Java
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        break;
    }
}
```

👉 Outer loop continues

💡 Pro Tip (Interview Gold)

Say this:

“Use **break** to exit loop, **continue** to skip iteration, and labeled statements to control outer loops in nested scenarios.”

If you want next:

- ✓ Pattern programs (very important 🔥)
- ✓ Tricky output questions
- ✓ Java interview rapid-fire

Just tell me 👍

Arrays — 1D, 2D, Arrays utility class
Explain with examples and most commonly asked interview questions

Here's a **complete + interview-focused guide on Java Arrays** 🙌

◆ 1. What is an Array?

👉 An array is a **fixed-size collection of elements of the same type** stored in **contiguous memory**.

✓ Example

```
Java
int[] arr = {10, 20, 30};
```

◆ 2. 1D Array (Single Dimension)

✓ Declaration + Initialization

```
Java
int[] arr = new int[3]; // size = 3
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;
```

✓ Shortcut Initialization

```
Java
int[] arr = {10, 20, 30};
```

✓ Traversal

```
Java
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

✓ Using for-each

```
Java
for (int num : arr) {
    System.out.println(num);
}
```

◆ 3. 2D Array (Matrix)

👉 Array of arrays

✓ Declaration

```
Java
int[][] matrix = new int[2][3];
```

✓ Initialization

```
Java
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};
```

✓ Traversal

```
Java
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
```

👉 Output:

```
1 2 3
4 5 6
```

🔥 Jagged Array (Important)

👉 Rows can have different sizes

```
Java
int[][] arr = {
    {1, 2},
    {3, 4, 5},
    {6}
};
```

◆ 4. Arrays Utility Class (java.util.Arrays)

👉 Provides useful methods for array operations

Use:

```
Java
import java.util.Arrays;
```

✓ 1. Sorting

```
Java
```

```
int[] arr = {5, 2, 8, 1};
Arrays.sort(arr);
System.out.println(Arrays.toString(arr));
```

👉 Output: [1, 2, 5, 8]

✓ 2. toString()

```
Java
System.out.println(Arrays.toString(arr));
```

👉 Converts array to readable string

✓ 3. equals()

```
Java
int[] a = {1, 2};
int[] b = {1, 2};

System.out.println(Arrays.equals(a, b)); // true
```

✓ 4. fill()

```
Java
int[] arr = new int[5];
Arrays.fill(arr, 10);
```

👉 Output: [10, 10, 10, 10, 10]

✓ 5. binarySearch()

```
Java
int[] arr = {1, 2, 3, 4};
int index = Arrays.binarySearch(arr, 3);
```

👉 Output: 2

⚠️ Array must be sorted!

🔥 Difference: Array vs ArrayList

Feature	Array	ArrayList
Size	Fixed	Dynamic
Speed	Faster	Slightly slower

Feature	Array	ArrayList
Type	Primitive + Object	Only Objects

Most Asked Interview Questions

? 1. What is difference between 1D and 2D array?

👉 1D → single list

👉 2D → table (rows & columns)

? 2. What is default value of array?

Type	Default
int	0
boolean	false
String	null

? 3. Can array size be changed?

👉 ❌ No (fixed size)

? 4. Difference between == and Arrays.equals()?

Java

```
int[] a = {1,2};
int[] b = {1,2};
```

```
System.out.println(a == b); // false
System.out.println(Arrays.equals(a,b)); // true
```

👉 == → reference comparison

👉 equals() → content comparison

? 5. What is jagged array?

👉 Array where rows have different sizes

? 6. Why array index starts from 0?

👉 Because memory address calculation is faster
(base address + index * size)

? 7. What happens if index is out of bound?

👉 Runtime error:

`ArrayIndexOutOfBoundsException`

? 8. How to copy array?

```
Java  
int[] copy = Arrays.copyOf(arr, arr.length);
```

? 9. Can we store different data types?

👉 ❌ No (same type only)

? 10. How arrays are stored in memory?

👉 Contiguous memory locations

💡 Pro Interview Tip

Say this confidently:

"Arrays are fixed-size, index-based structures stored in contiguous memory.
Use `Arrays` utility class for operations like sorting, searching, and comparison."

🚀 Bonus (Very Important Coding Questions)

You should practice:

- Reverse an array
- Find max/min
- Remove duplicates
- Rotate array
- 2D matrix traversal

- Transpose matrix

If you want next level:

- ✓ Top 20 array interview questions with solutions
- ✓ Pattern + DSA level questions
- ✓ Real company questions (TCS, Accenture, Infosys)

Just tell me 👍

String — immutability, string pool

Here's a **clear + interview-focused explanation of Java Strings**, especially **immutability** and **string pool** 👉

♦ What is a String in Java?

👉 A **String** is an **object** in Java used to store text.

```
Java
String name = "Lawrence";
```

👉 Internally, **String** is a class from **java.lang.String**

🔥 1. String Immutability

👉 **Immutability** = cannot be changed once created

✓ Example

```
Java
String s = "Hello";
s.concat(" World");

System.out.println(s);
```

👉 Output:

Hello

🤖 Why?

Because **concat()** creates a **new object**, it does NOT modify the original.

✓ Correct Way

```
Java
```

```
String s = "Hello";
s = s.concat(" World");

System.out.println(s);
```

👉 Output:

```
Hello World
```

🔥 Internal Working

```
Java
String s1 = "Hello";
String s2 = s1.concat(" World");
```

👉 Memory:

- "Hello" → remains unchanged
- "Hello World" → new object created

💡 Why Strings are Immutable?

👉 Key reasons:

- Security (used in URLs, DB connections)
- Thread-safe
- String pool optimization
- Caching (hashcode)

🔥 2. String Pool (String Constant Pool)

👉 Special memory area inside heap where **duplicate strings are avoided**

✅ Example

```
Java
String s1 = "Java";
String s2 = "Java";
```

👉 Both refer to **same object** in pool

```
Java
System.out.println(s1 == s2); // true
```

🧠 Diagram (Concept)

- Only **one** "Java" stored in pool

- Both variables point to same reference

🔥 3. new Keyword vs Literal

✅ Using Literal

```
Java
String s1 = "Java";
String s2 = "Java";
```

👉 Stored in **string pool**

! Using new

```
Java
String s3 = new String("Java");
```

👉 Creates:

- One object in pool ("Java")
- One object in heap (new object)

```
Java
System.out.println(s1 == s3); // false
```

🔥 4. equals() vs == (VERY IMPORTANT 🔥)

✅ Example

```
Java
String a = "Java";
String b = new String("Java");

System.out.println(a == b); // false
System.out.println(a.equals(b)); // true
```

👉 Difference:

Operator	Checks
==	reference
equals()	content

🔥 5. String Interning

👉 Forces string into pool

```
Java
String s1 = new String("Java");
String s2 = s1.intern();

System.out.println(s2 == "Java"); // true
```

Most Asked Interview Questions

? 1. Why String is immutable?

👉 Answer:

- Security
- Thread safety
- Performance (string pool reuse)

? 2. What is String Pool?

👉 A memory area where duplicate strings are stored once to save memory

? 3. Difference between String and StringBuilder?

👉 String → immutable

👉 StringBuilder → mutable (faster for changes)

? 4. What happens when we do String s = "Hello" + "World"?

👉 Compile-time optimization → "HelloWorld" stored in pool

? 5. What about runtime concatenation?

```
Java
String a = "Hello";
String b = "World";

String c = a + b;
```

👉 New object created in heap

? 6. How many objects are created?

Java

```
String s = new String("Java");
```

👉 2 objects:

- Pool → "Java"
- Heap → new object

? 7. Is String thread-safe?

👉 Yes (because immutable)

? 8. Can we modify String?

👉 ❌ No (must create new one)

💡 Pro Interview Tip

Say this confidently:

"String is immutable in Java, which enables security, thread safety, and efficient memory usage via the string pool. Any modification creates a new object."

🚀 Bonus (Important for Interviews)

Use:

- **StringBuilder** → single-threaded (fast)
- **StringBuffer** → multi-threaded (safe)

If you want next:

- ✓ Top tricky String interview questions
- ✓ Output-based questions (very common 🔥)
- ✓ Coding problems (reverse string, palindrome, etc.)

Just tell me 👍

StringBuilder vs StringBuffer vs String

Here's a **clear + interview-ready comparison of String, StringBuilder, and StringBuffer in Java** 👉

◆ 1. String (Immutable)

👉 From `java.lang.String`

✓ Key Property

- **Immutable** → cannot change once created

✓ Example

```
Java
String s = "Hello";
s.concat(" World");

System.out.println(s); // Hello
```

👉 New object is created, original stays same

💡 Use When

- Data should not change
- Security matters
- String literals / constants

◆ 2. StringBuilder (Mutable, Fast 🚀)

👉 From `java.lang.StringBuilder`

✓ Key Property

- **Mutable** → can modify without creating new object
- **Not thread-safe**

✓ Example

```
Java
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");

System.out.println(sb); // Hello World
```

💡 Use When

- Frequent modifications
- Single-threaded applications
- Performance is important

♦ 3. StringBuffer (Mutable, Thread-Safe)

 From `java.lang.StringBuffer`

✓ Key Property

- Mutable
- Thread-safe (synchronized)

✓ Example

```
Java
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");

System.out.println(sb); // Hello World
```

Use When

- Multi-threaded environment
- Need thread safety

Key Differences (Very Important)

Feature	String	StringBuilder	StringBuffer
Mutability	✗ Immutable	✓ Mutable	✓ Mutable
Thread-safe	✓ Yes	✗ No	✓ Yes
Performance	Slow (new object)	Fast 	Slower (sync overhead)
Memory	More usage	Efficient	Efficient
Introduced	Java 1.0	Java 5	Java 1.0

Performance Example (Interview Favorite)

```
Java
String s = "";
for (int i = 0; i < 1000; i++) {
    s += i; // creates new object every time ✗
}
```

 Slow ✗

✓ Better Approach

```
Java
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append(i); // modifies same object ✓
}
```

👉 Fast 🚀

🎯 Most Asked Interview Questions

? 1. Difference between String, StringBuilder, and StringBuffer?

👉 Answer:

- **String** → immutable
- **StringBuilder** → mutable, fast, not thread-safe
- **StringBuffer** → mutable, thread-safe

? 2. Which is faster?

👉 Fastest → **StringBuilder**

👉 Slowest → **String** (due to immutability)

? 3. Why StringBuilder is faster than StringBuffer?

👉 Because:

- No synchronization overhead
- Not thread-safe → less locking

? 4. When to use StringBuffer?

👉 Only when:

- Multiple threads modifying same string

? 5. Is String thread-safe?

👉 Yes (immutable)

? 6. Can `StringBuilder` be used in multi-threading?

👉 ❌ Not safe
(need external synchronization)

? 7. What happens internally in String concatenation?

```
Java
String s = "A" + "B";
```

👉 Compile-time optimization → "AB"

```
Java
String a = "A";
String b = "B";
String c = a + b;
```

👉 Uses `StringBuilder` internally

💡 Pro Interview Tip (🔥 Say this)

"Use `String` for constants, `StringBuilder` for performance in single-threaded cases, and `StringBuffer` only when thread safety is required."

🚀 Real-World Insight

👉 In most modern applications:

- **`StringBuilder` is used 90% of the time**
- `StringBuffer` is rarely used unless explicitly needed

If you want next level prep:

- ✓ Tricky output questions (very common in interviews)
- ✓ Coding problems using strings
- ✓ Internal memory diagrams

Just tell me 👉

String methods — split, trim, replace, substring, indexOf, contains, format

Here's a **clean + interview-focused guide to important Java String methods** 👉

We'll cover:

👉 split, trim, replace, substring, indexOf, contains, format

(All belong to `java.lang.String`)

◆ 1. `split()` — Break string into parts

👉 Splits string using **regex (delimiter)**

✓ Example

```
Java
String s = "Java,Python,Go";
String[] arr = s.split(",");

for (String lang : arr) {
    System.out.println(lang);
}
```

👉 Output:

```
Java
Python
Go
```

⚠ Interview Trap

```
Java
"abc.def".split(".") // WRONG ❌
```

👉 `.` is regex → matches everything

✓ Correct:

```
Java
"abc.def".split("\\.")
```

◆ 2. `trim()` — Remove spaces

👉 Removes **leading and trailing spaces**

✓ Example

```
Java
String s = " Hello ";
System.out.println(s.trim());
```

👉 Output:

```
Hello
```

⚠ Note

- Does NOT remove spaces in middle

◆ 3. replace() — Replace characters/strings

✓ Example

```
Java
String s = "Java is fun";
System.out.println(s.replace("Java", "Python"));
```

👉 Output:

```
Python is fun
```

✓ replace vs replaceAll

Method	Uses
replace()	normal text
replaceAll()	regex

⚠ Example

```
Java
s.replaceAll(".", "*"); // replaces everything
```

◆ 4. substring() — Extract part

✓ Syntax

```
Java
substring(start)
substring(start, end)
```

✓ Example

```
Java
String s = "HelloWorld";

System.out.println(s.substring(5)); // World
System.out.println(s.substring(0, 5)); // Hello
```

⚠ Rule

- start inclusive

- end exclusive

◆ 5. indexOf() — Find position

✓ Example

```
Java
String s = "Java Programming";

System.out.println(s.indexOf("a"));    // 1
System.out.println(s.indexOf("a", 2)); // 3
```

💡 If not found

```
Java
s.indexOf("z"); // -1
```

◆ 6. contains() — Check existence

✓ Example

```
Java
String s = "Java Programming";

System.out.println(s.contains("Java")); // true
System.out.println(s.contains("Python")); // false
```

◆ 7. format() — Format string

👉 Similar to printf

✓ Example

```
Java
String name = "John";
int age = 25;

String result = String.format("Name: %s, Age: %d", name, age);
System.out.println(result);
```

👉 Output:

```
Name: John, Age: 25
```

💡 Common Format Specifiers

Specifier	Meaning
%s	String
%d	Integer
%f	Float
%.2f	2 decimal

✓ Example

Java

```
double price = 99.456;
```

```
System.out.println(String.format("%.2f", price));
```

👉 Output:

```
99.46
```

🔥 Summary Table

Method	Purpose
split()	Break string
trim()	Remove spaces
replace()	Replace text
substring()	Extract part
indexOf()	Find position
contains()	Check existence
format()	Format output

🎯 Most Asked Interview Questions

? 1. Difference between replace() and replaceAll()?

👉 replace → normal text

👉 replaceAll → regex

? 2. What does split() use internally?

👉 Regular Expressions (regex)

? 3. What if substring index is invalid?

👉 Runtime error:

`StringIndexOutOfBoundsException`

? 4. What does indexOf return if not found?

👉 -1

? 5. Is contains() case-sensitive?

👉 ✅ Yes

```
Java
"Java".contains("java") // false
```

? 6. Can trim remove all spaces?

👉 ❌ Only start & end

? 7. Which method is used for formatted output?

👉 `String.format()` or `System.out.printf()`

💡 Pro Interview Tip

Say this:

"Most String methods return new objects since String is immutable. Methods like split use regex, substring uses index ranges, and format is used for structured output."

🚀 Bonus Practice (Very Important 🔥)

Try coding:

- Count words using `split()`
- Reverse string using `substring()`
- Remove extra spaces using `trim()`
- Find substring using `indexOf()`

If you want next:

- ✓ Tricky output-based questions (very common)
- ✓ Real coding problems using strings
- ✓ Java mini test

Just tell me 👍

Type casting — implicit (widening) vs explicit (narrowing)

Here's a **clear + interview-focused explanation of Type Casting in Java** 👉

♦ What is Type Casting?

👉 Type casting means **converting one data type into another**

🔥 1. Implicit Casting (Widening)

- 👉 Done **automatically by Java**
- 👉 Converting **smaller** → **larger data type**

✓ Order (Important 🔥)

byte → short → int → long → float → double

✓ Example

```
Java
int a = 10;
double b = a; // automatic conversion

System.out.println(b); // 10.0
```

💡 Why no data loss?

- 👉 Because larger type can store smaller type safely

🔥 2. Explicit Casting (Narrowing)

- 👉 Done **manually by programmer**
- 👉 Converting **larger** → **smaller data type**

✓ Syntax

Java

```
datatype var = (datatype) value;
```

✓ Example

```
Java
double a = 10.5;
int b = (int) a;

System.out.println(b); // 10
```

⚠ Data Loss

👉 Decimal part removed

🔥 Another Example (Overflow ⚠)

```
Java
int x = 130;
byte y = (byte) x;

System.out.println(y);
```

👉 Output:

```
-126
```

👉 Because byte range = -128 to 127

🔥 Real Meaning

Type	Conversion	Who does it	Risk
Widening	small → large	Java	No loss
Narrowing	large → small	Programmer	Data loss possible

🔥 Special Case (char ↔ int)

```
Java
char ch = 'A';
int num = ch;

System.out.println(num); // 65
```

```
Java
int x = 66;
char c = (char) x;

System.out.println(c); // B
```

Type Promotion in Expressions

Java

```
byte a = 10;
byte b = 20;

int result = a + b; // auto promoted to int
```

👉 Small types → converted to `int` during operations

Most Asked Interview Questions

? 1. Difference between widening and narrowing?

👉 Answer:

- Widening → automatic, safe
- Narrowing → manual, may lose data

? 2. Why explicit casting required?

👉 To prevent accidental data loss

? 3. Can we cast boolean?

👉 ❌ No (boolean cannot be converted)

? 4. What is type promotion?

👉 Smaller types automatically converted to `int` in expressions

? 5. What happens in this?

Java

```
int a = 5;
double b = 2;

System.out.println(a / b);
```

- 👉 Output: 2.5
- 👉 Because int → double (widening)

? 6. Tricky Question 🔥

```
Java
int a = 10;
int b = 3;

System.out.println(a / b);
```

- 👉 Output: 3 (not 3.33 ❌)
- 👉 Because both are int → integer division

✅ Fix

```
Java
System.out.println((double)a / b); // 3.33
```

? 7. Can narrowing cause overflow?

- 👉 ✅ Yes (as seen in byte example)

💡 Pro Interview Tip

Say this confidently:

“Widening casting is automatic and safe, while narrowing casting is explicit and may cause data loss or overflow.”

🚀 Bonus (Very Important)

🔥 Memory View

Type	Size
byte	1 byte
short	2 bytes
int	4 bytes
long	8 bytes
float	4 bytes
double	8 bytes

If you want next:

- ✓ Tricky output questions (very common in interviews)
- ✓ Mini test for practice
- ✓ Java interview rapid-fire

Just tell me 👍

Wrapper classes — Integer, Double, Character, etc.
Autoboxing & Unboxing

Here's a **clear + interview-focused explanation of Wrapper Classes + Autoboxing & Unboxing in Java**



♦ What are Wrapper Classes?

👉 Wrapper classes are **object representations of primitive data types**

✓ Examples

Primitive	Wrapper Class
int	java.lang.Integer
double	java.lang.Double
char	java.lang.Character
boolean	java.lang.Boolean
long	java.lang.Long

🔥 Why Wrapper Classes?

👉 Because Java works with **objects**, not primitives, in many cases:

💡 Use Cases

- Collections (**ArrayList**, **HashMap**)
- Generics
- Utility methods (parsing, conversions)

♦ 1. Autoboxing (Primitive → Object)

👉 Automatic conversion by Java

✓ Example

```
Java
int a = 10;

Integer obj = a; // autoboxing
```

👉 Internally:

```
Java
Integer obj = Integer.valueOf(a);
```

♦ 2. Unboxing (Object → Primitive)

👉 Automatic conversion back

✓ Example

```
Java
Integer obj = 20;

int num = obj; // unboxing
```

👉 Internally:

```
Java
int num = obj.intValue();
```

🔥 Manual Boxing & Unboxing

✓ Boxing

```
Java
int a = 5;
Integer obj = Integer.valueOf(a);
```

✓ Unboxing

```
Java
Integer obj = 10;
int x = obj.intValue();
```

🔥 Important Wrapper Methods

✓ 1. Parsing

```
Java
int num = Integer.parseInt("123");
double d = Double.parseDouble("10.5");
```

✓ 2. valueOf()

Java

```
Integer obj = Integer.valueOf("123");
```

✓ 3. toString()

Java

```
Integer obj = 100;
String s = obj.toString();
```

🔥 Important Concept: Caching (Very Important 🔥)

Java

```
Integer a = 100;
Integer b = 100;

System.out.println(a == b); // true
```

👉 Because values between -128 to 127 are cached

Java

```
Integer x = 200;
Integer y = 200;

System.out.println(x == y); // false
```

🔥 equals() vs == (Wrapper)

Java

```
Integer a = 100;
Integer b = 100;

System.out.println(a.equals(b)); // true
```

👉 Always use equals() for comparison

⚠ Common Pitfall (NullPointerException)

Java

```
Integer obj = null;
int x = obj; // ❌ Runtime error
```

👉 Unboxing null → **NullPointerException**

🎯 Most Asked Interview Questions

? 1. What are wrapper classes?

👉 Object form of primitive data types

? 2. Why needed?

👉 Because:

- Collections work only with objects
- Required for generics

? 3. What is autoboxing?

👉 Automatic primitive → object conversion

? 4. What is unboxing?

👉 Object → primitive conversion

? 5. Difference between parseInt() and valueOf()?

Method	Returns
parseInt()	primitive int
valueOf()	Integer object

? 6. Why Integer caching?

👉 To improve performance & memory

? 7. Can wrapper be null?

👉 ☒ Yes

👉 Primitives → ☒ cannot be null

? 8. Which is faster: primitive or wrapper?

👉 Primitive (no object overhead)

? 9. What happens here?

Java

```
Integer a = null;
System.out.println(a + 5);
```

👉 ❌ NullPointerException (unboxing happens)

💡 Pro Interview Tip

Say this confidently:

“Wrapper classes convert primitives into objects, enabling their use in collections and generics. Autoboxing and unboxing simplify conversions, but developers must handle null values carefully to avoid exceptions.”

🚀 Real-World Insight

👉 In real projects:

- Use **primitive** for performance
- Use **wrapper** when working with:
 - Collections
 - APIs
 - Nullable values

If you want next:

- ✅ Tricky MCQs (very common in TCS/Accenture)
- ✅ Output-based questions
- ✅ Mini mock interview

Just tell me 👍

var keyword (Java 10+)
Explain with interview questions

Here's a **clear + interview-focused explanation of var (Java 10+)** 👉

♦ What is var in Java?

👉 **var** is used for **local variable type inference**

👉 Java automatically **detects the type at compile time**

✅ Example

```
Java
var x = 10;           // int
var name = "Java";    // String
var price = 99.5;     // double
```

👉 Compiler internally treats it as:

```
Java
int x = 10;
String name = "Java";
double price = 99.5;
```

🔥 Important Point

👉 var is **NOT** a keyword

👉 It's a **reserved type name** (since Java 10)

♦ Where can we use var?

✅ Allowed:

- Local variables inside methods
- Loops
- Enhanced for loop

✅ Example (loop)

```
Java
for (var i = 0; i < 5; i++) {
    System.out.println(i);
}
```

✅ Example (for-each)

```
Java
int[] arr = {1, 2, 3};

for (var num : arr) {
    System.out.println(num);
}
```

● Where var is NOT allowed?

- ✗ Instance variables
- ✗ Method parameters
- ✗ Return types
- ✗ Without initialization

✗ Invalid Examples

```
Java
var x;           // ✗ no initialization
var a = null;    // ✗ cannot infer type

class Test {
    var name = "Java"; // ✗ not allowed (instance variable)
}
```

🔥 Type Inference Rules

👉 Type is decided **at compile time**, not runtime

```
Java
var x = 10; // int
x = 20;     // ✓ OK
x = "Hi";   // ✗ Error
```

🔥 Complex Example

```
Java
var list = new ArrayList<String>();
```

👉 Type inferred as:

```
Java
ArrayList<String> list
```

🔥 Benefits of var

- ✓ Reduces boilerplate code
- ✓ Improves readability (sometimes)
- ✓ Useful with complex generics

⚠ When NOT to use var

- ✗ When type is unclear
- ✗ Reduces readability

🚫 Bad Example

```
Java
var x = getData(); // unclear type
```

✓ Good Example

```
Java
List<String> list = getData(); // clear
```

🔥 Difference: var vs Object

Feature	var	Object
Type decided	Compile-time	Runtime
Type safety	Strong	Weak
Casting needed	No	Yes

✓ Example

```
Java
var x = 10; // int
Object y = 10; // Object
```

🎯 Most Asked Interview Questions

? 1. Is var dynamically typed?

- ✗ No
- ✗ It's statically typed (compile-time inference)

? 2. Can we use var without initialization?

- ✗ No

? 3. Can var be null?

- ✗ No (type cannot be inferred)

? 4. Can we use var for class variables?

👉 ❌ No (only local variables)

? 5. Is var a keyword?

👉 ❌ No (reserved type name)

? 6. Does var improve performance?

👉 ❌ No

👉 It's only syntactic sugar

? 7. What is inferred type here?

```
Java
var x = 10.0;
```

👉 double

? 8. Can we change type later?

👉 ❌ No (type is fixed)

? 9. Tricky Question 🔥

```
Java
var list = List.of(1, 2, 3);
```

👉 Type = List<Integer>

? 10. Can we use with lambda?

👉 Yes (Java 11+)

```
Java
(var x, var y) -> x + y
```

💡 Pro Interview Tip

Say this confidently:

“**var** provides local variable type inference in Java. It improves readability in some cases but should be used carefully to avoid reducing code clarity.”

Real-World Insight

👉 In real projects:

- Use **var** for:
 - Obvious types (`new ArrayList<>()`)
- Avoid for:
 - Complex or unclear return types

If you want next:

- ✓ OOP concepts (very important 🔥)
- ✓ Java interview rapid-fire
- ✓ Real coding questions

Just tell me 👍

Input/Output — Scanner, BufferedReader

Here's a **clear + interview-focused explanation of Java Input/Output using Scanner and BufferedReader** 👉

♦ 1. Scanner (Easy & Beginner-Friendly)

👉 From **java.util.Scanner**

👉 Used to take input from:

- Keyboard
- Files
- Streams

✓ Example

```
Java
import java.util.Scanner;

Scanner sc = new Scanner(System.in);

int num = sc.nextInt();
String name = sc.next();
```

```
System.out.println(num);
System.out.println(name);
```

✓ Common Methods

Method	Use
<code>nextInt()</code>	read int
<code>nextDouble()</code>	read double
<code>next()</code>	read single word
<code>nextLine()</code>	read full line

⚠ Common Issue (VERY IMPORTANT 🔥)

```
Java
int x = sc.nextInt();
String s = sc.nextLine(); // ❌ skipped
```

👉 Fix:

```
Java
sc.nextLine(); // consume leftover newline
String s = sc.nextLine();
```

💡 Pros

- Easy to use
- Built-in parsing

❌ Cons

- Slower (uses regex internally)

♦ 2. BufferedReader (Fast ⚡)

👉 From `java.io.BufferedReader`

👉 Reads input as **text (String)**

✓ Example

```
Java
import java.io.BufferedReader;
import java.io.InputStreamReader;

BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

String line = br.readLine();
System.out.println(line);
```

Important

 Always returns **String**

```
Java
int num = Integer.parseInt(br.readLine());
```

Pros

- Faster than Scanner
- Good for competitive programming

Cons

- Manual parsing required
- More code

Difference: Scanner vs BufferedReader

Feature	Scanner	BufferedReader
Speed	Slow	Fast 
Parsing	Built-in	Manual
Ease	Easy	Slightly complex
Input type	Primitive + String	Only String

When to Use What?

 Use **Scanner**:

- Beginners
- Small programs
- Interviews

 Use **BufferedReader**:

- Large input data
- Competitive coding
- Performance-critical apps

Most Asked Interview Questions

? 1. Difference between Scanner and BufferedReader?

👉 Answer:

- Scanner → easy but slow
- BufferedReader → fast but manual parsing

? 2. Why Scanner is slow?

👉 Uses regex + parsing internally

? 3. Why BufferedReader is faster?

👉 Reads raw text without parsing

? 4. Can BufferedReader read int directly?

👉 ❌ No

👉 Must use `Integer.parseInt()`

? 5. Which is better for competitive programming?

👉 BufferedReader (faster)

? 6. What happens if we don't close Scanner?

👉 Resource leak (best practice to close)

```
Java
sc.close();
```

? 7. Can Scanner read full line?

👉 Yes → `nextLine()`

? 8. Tricky Question 🔥

Java

```
Scanner sc = new Scanner(System.in);

int x = sc.nextInt();
String s = sc.nextLine();

System.out.println(s);
```

👉 Output: empty string ❌

👉 Reason: newline issue

💡 Pro Interview Tip

Say this confidently:

“Scanner is convenient and easy to use, but slower due to internal parsing. BufferedReader is faster and preferred when performance matters, but requires manual conversion.”

🚀 Real-World Insight

👉 In real projects:

- Use **Scanner** for quick input
- Use **BufferedReader** when handling **large data or high performance systems**

If you want next:

- ✓ Exception Handling (very important 🔥)
- ✓ File handling
- ✓ Java interview rapid-fire

Just tell me 👍

Command-line arguments

Here's a **clear + interview-focused explanation of Command-Line Arguments in Java** 👉

💠 What are Command-Line Arguments?

👉 They are **inputs passed to a Java program at runtime** via the command line

👉 Received in:

Java

```
public static void main(String[] args)
```

👉 args is an **array of Strings**

◆ Basic Example

✓ Code

```
Java
class Test {
    public static void main(String[] args) {
        System.out.println(args[0]);
        System.out.println(args[1]);
    }
}
```

✓ Run Command

```
Bash
java Test Hello Java
```

👉 Output

```
Hello
Java
```

🔥 Key Points

- `args` → array of **String**
- Inputs are **space-separated**
- Index starts from 0

◆ Example with Loop

```
Java
public class Demo {
    public static void main(String[] args) {
        for (int i = 0; i < args.length; i++) {
            System.out.println(args[i]);
        }
    }
}
```

◆ Passing Numbers (Important)

👉 Even numbers are received as **String**

✓ Example

```
Java
public class Sum {
    public static void main(String[] args) {
        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);
    }
}
```

```
        System.out.println(a + b);
    }
}
```

Run:

Bash

```
java Sum 10 20
```

👉 Output:

```
30
```

🔥 What happens if no arguments?

Java

```
System.out.println(args[0]); // ❌
```

👉 Error:

```
ArrayIndexOutOfBoundsException
```

💠 Safe Handling

Java

```
if (args.length > 0) {
    System.out.println(args[0]);
} else {
    System.out.println("No arguments passed");
}
```

🔥 Real Use Cases

👉 Used in:

- Running programs with input
- Automation scripts
- Configuration settings

🎯 Most Asked Interview Questions

? 1. What is String[] args?

👉 Array storing command-line inputs as strings

? 2. Can we change name args?

👉 ✅ Yes

```
Java
public static void main(String[] data)
```

? 3. Can we pass multiple arguments?

👉 ✅ Yes (space-separated)

? 4. Are arguments always String?

👉 ✅ Yes

👉 Must convert manually if needed

? 5. What if no arguments passed?

👉 `args.length = 0`

? 6. Can we overload main method?

👉 ✅ Yes

But JVM only calls:

```
Java
public static void main(String[] args)
```

? 7. Difference between Scanner and command-line arguments?

Feature	Scanner	Command-line
Input time	Runtime (interactive)	Program start
Flexibility	High	Limited
Use case	User input	Automation

? 8. Can we pass special characters?

👉 ✅ Yes (use quotes if needed)

Bash

```
java Test "Hello World"
```

? 9. Where are arguments stored?

👉 In **main method parameter array**

💡 Pro Interview Tip

Say this confidently:

“Command-line arguments are passed at runtime and stored as a String array in the main method. They are useful for automation and require manual parsing for other data types.”

🚀 Bonus Practice

Try:

- Sum of numbers using args
- Find max from args
- Reverse arguments
- Count number of arguments

If you want next:

- ✅ Exception Handling (very important 🔥)
- ✅ File handling
- ✅ Java mini project ideas

Just tell me 👍

Stack memory vs heap memory explain with example

Here's a **clear + interview-ready explanation of Stack vs Heap Memory in Java** 👉

💠 What is Memory in Java?

👉 Java mainly uses **two memory areas**:

- **Stack Memory**
- **Heap Memory**

◆ 1. Stack Memory (📁)

👉 Used for:

- Method calls
- Local variables
- Function execution

✅ Example

```
Java
public class Demo {
    public static void main(String[] args) {
        int x = 10;
        int y = 20;
        add(x, y);
    }

    static void add(int a, int b) {
        int sum = a + b;
        System.out.println(sum);
    }
}
```

🧠 How Stack Works

- `main()` pushed to stack
- Variables `x, y` stored
- `add()` called → new stack frame created
- After execution → removed automatically

💡 Key Features

- Stores **primitive variables**
- Stores **method calls (stack frames)**
- **Fast access**
- **LIFO (Last In First Out)**

◆ 2. Heap Memory (🧠)

👉 Used for:

- Objects
- Instance variables

✅ Example

```
Java
class Person {
    String name;
}

public class Test {
    public static void main(String[] args) {
        Person p = new Person();
        p.name = "John";
    }
}
```

How Heap Works

- Object **Person** created in **heap**
- Reference **p** stored in **stack**
- Heap managed by **Java Garbage Collector**

Key Features

- Stores **objects**
- Shared across methods
- Slower than stack
- Uses **Garbage Collection**

Stack vs Heap (Important Table)

Feature	Stack	Heap
Stores	Local variables, method calls	Objects
Access speed	Fast ⚡	Slower
Size	Smaller	Larger
Memory management	Automatic (function scope)	Garbage Collector
Thread safety	Thread-specific	Shared
Structure	LIFO	No specific order

Visual Understanding

STACK (Thread-wise)

```
main()
x = 10
y = 20
p → reference
```

```
add()
a = 10
b = 20
sum = 30
```



```
-----
HEAP (Shared)
-----
```

```
Person object
name = "John"
-----
```

🔥 Key Concept (Very Important 🔥)

Java

```
Person p = new Person();
```

👉 Breakdown:

- `new Person()` → object in **heap**
- `p` → reference in **stack**

🔥 Common Errors

❌ StackOverflowError

👉 Infinite recursion

Java

```
void fun() {
    fun(); // infinite calls
}
```

❌ OutOfMemoryError

👉 Heap memory full

🎯 Most Asked Interview Questions

? 1. Difference between stack and heap?

👉 Answer:

- Stack → stores local variables and method calls
- Heap → stores objects

? 2. Where are objects stored?

👉 Heap memory

? 3. Where are references stored?

👉 Stack (for local variables)

? 4. Why stack is faster?

👉 Because:

- Simple memory allocation
 - LIFO structure
-

? 5. Who manages heap memory?

👉 Garbage Collector

? 6. Can heap memory be accessed by multiple threads?

👉 ☒ Yes (shared)

? 7. What causes StackOverflowError?

👉 Too many method calls (recursion)

? 8. What happens when object is not referenced?

👉 Garbage collected

💡 Pro Interview Tip

Say this confidently:

“Stack memory stores method calls and local variables with fast access, while heap memory stores objects and is managed by the garbage collector.”

🚀 Real-World Insight

👉 In real projects:

- Too many objects → heap issues
- Deep recursion → stack overflow

If you want next:

- ✓ JVM architecture (very important 🔥)
- ✓ Garbage Collection deep dive
- ✓ Java memory interview questions

Just tell me 👍

Explain Garbage collection in java also give most commonly asked interview questions

Here's a **clear + interview-ready explanation of Garbage Collection (GC) in Java** 👉

♦ What is Garbage Collection?

- 👉 Garbage Collection is the process of **automatically deleting unused objects from memory (heap)**
- 👉 Managed by the **Java Garbage Collector**

♦ Why Garbage Collection?

👉 To:

- Free unused memory
- Prevent memory leaks
- Improve performance

♦ Simple Example

```
Java
class Demo {
    public static void main(String[] args) {
        Demo d1 = new Demo();
        Demo d2 = new Demo();

        d1 = null; // object eligible for GC
    }
}
```

👉 Object created by **d1** is now **eligible for GC**

🔥 Important Point

- 👉 GC does NOT destroy object immediately
- 👉 It only makes them **eligible**, JVM decides when to clean

◆ When Object Becomes Eligible for GC?

✓ 1. Null Reference

```
Java
obj = null;
```

✓ 2. Reassigning Reference

```
Java
obj1 = new Demo();
obj1 = new Demo(); // old object eligible
```

✓ 3. Anonymous Object

```
Java
new Demo(); // no reference → eligible
```

✓ 4. Local Scope Ends

```
Java
void method() {
    Demo d = new Demo();
} // d destroyed → eligible
```

✓ 5. Island of Isolation (Tricky 🔥)

```
Java
class Test {
    Test t;
}

Test t1 = new Test();
Test t2 = new Test();

t1.t = t2;
t2.t = t1;

t1 = null;
t2 = null;
```

👉 Both objects reference each other but are unreachable → GC

◆ How GC Works (Simplified)

👉 JVM uses algorithms like:

- Mark → identify reachable objects
- Sweep → remove unused

- Compact → optimize memory

◆ Types of Garbage Collectors (Basic Idea)

👉 JVM has different collectors:

- Serial GC
- Parallel GC
- G1 GC (modern default)

◆ finalize() Method (Deprecated ⚠)

```
Java
protected void finalize() {
    System.out.println("Cleanup");
}
```

👉 Called before GC (not guaranteed)

⚠ Depreciated → avoid using

◆ System.gc()

```
Java
System.gc();
```

👉 Requests JVM to run GC

👉 ! Not guaranteed

🔥 Key Concepts

- GC works only on **heap memory**
- Objects must be **unreachable**
- JVM decides timing
- Cannot force GC

🎯 Most Asked Interview Questions

? 1. What is Garbage Collection?

👉 Automatic memory management that removes unused objects from heap

? 2. When is object eligible for GC?

👉 When it becomes unreachable

? 3. Can we force GC?

👉 ❌ No

👉 Only request using `System.gc()`

? 4. What is difference between delete and GC?

👉 Java has no `delete`

👉 GC handles memory automatically

? 5. Does GC run immediately after null?

👉 ❌ No (JVM decides)

? 6. What is memory leak in Java?

👉 Objects not used but still referenced

? 7. What is finalize()?

👉 Method called before GC (deprecated)

? 8. What is island of isolation?

👉 Objects referencing each other but unreachable

? 9. Does GC clean stack memory?

👉 ❌ No

👉 Only heap memory

? 10. Which part of JVM handles GC?

👉 Garbage Collector

💡 Pro Interview Tip

Say this confidently:

“Garbage Collection in Java automatically removes unreachable objects from heap memory. It improves memory management but runs at JVM’s discretion, not manually.”

🚀 Real-World Insight

👉 In real projects:

- Avoid unnecessary object creation
- Remove unused references
- Use proper data structures

If you want next:

- ✅ JVM architecture (very important 🔥)
- ✅ Memory leaks deep dive
- ✅ Java interview rapid-fire

Just tell me 👍

Difference between Java 8 17 21 and other latest interview questions

For **2 years of Java experience**, interviewers usually don't expect you to memorize every Java version. They expect you to know the **major changes in Java 8, 11, 17, and 21**, because these are the most widely used LTS (Long-Term Support) releases.

Java Version Timeline

Version	Release	Importance
Java 8	2014	Most revolutionary release
Java 11	2018	LTS
Java 17	2021	Current enterprise favorite
Java 21	2023	Latest LTS widely being adopted

🔥 Java 8 (Most Important)

Java 8 completely changed how Java is written.

1. Lambda Expressions

Before Java 8:

```
Java
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("Running");
    }
};
```

Java 8:

```
Java
Runnable r = () -> System.out.println("Running");
```

2. Functional Interfaces

```
Java
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}
```

Examples:

- Predicate
- Function
- Consumer
- Supplier

3. Stream API

```
Java
List<Integer> nums = List.of(1,2,3,4,5);

nums.stream()
    .filter(n -> n % 2 == 0)
    .forEach(System.out::println);
```

Output:

```
2
4
```


4. Method References

```
Java
list.forEach(System.out::println);
```

instead of

```
Java
list.forEach(x -> System.out.println(x));
```

5. Optional Class

Avoids NullPointerException.

```
Java
Optional<String> name =
    Optional.ofNullable(userName);
```

6. New Date API

Package:

```
Java
java.time
```

Classes:

- `LocalDate`
- `LocalTime`
- `LocalDateTime`

Java 11

1. String Methods

```
Java
" ".isBlank();

"hello".repeat(3);

"hello".lines();
```

2. var in Lambda Parameters

```
Java
(var x, var y) -> x + y
```

3. HttpClient API

Old:

```
Java
URLConnection
```

New:

```
Java
HttpClient client = HttpClient.newHttpClient();
```

Java 17 (Very Common Interview Topic)

1. Sealed Classes

Control inheritance.

```
Java
public sealed class Vehicle
permits Car, Bike {
}
```

Only permitted classes can extend.

2. Pattern Matching for instanceof

Old:

```
Java
if(obj instanceof String) {
    String s = (String)obj;
}
```

Java 17:

```
Java
if(obj instanceof String s) {
    System.out.println(s.length());
}
```

3. Records

Reduces boilerplate.

Old:

```
Java
class Employee {
    private String name;
    private int age;

    // getters
    // setters
    // constructor
}
```

Java 17:

```
Java
record Employee(String name, int age) {}
```

Automatically provides:

- constructor
- getter methods
- equals()
- hashCode()
- toString()

Java 21 (Latest LTS)

1. Virtual Threads

Huge feature.

Traditional:

```
Java
Thread thread = new Thread(task);
```

Virtual Thread:

```
Java
Thread.startVirtualThread(task);
```

Allows millions of lightweight threads.

Used heavily in:

- Spring Boot 3
- Microservices

- High-concurrency systems

2. String Templates (Preview)

```
Java
String name = "Lawrence";

String msg =
    STR."Hello \{name}";
```

Similar to JavaScript template literals.

3. Pattern Matching for switch

```
Java
Object obj = "Hello";

switch(obj) {
    case String s -> System.out.println(s);
    case Integer i -> System.out.println(i);
    default -> {}
}
```

4. Record Patterns

```
Java
record Point(int x, int y) {}

if(obj instanceof Point(int x, int y)) {
    System.out.println(x);
}
```

Most Asked Java Version Interview Questions

Q1. What was the biggest feature introduced in Java 8?

Answer:

Lambda Expressions and Stream API.

Q2. What is a Functional Interface?

```
Java
@FunctionalInterface
interface Test {
```

```
void run();
}
```

Only one abstract method.

Q3. Difference between Collection and Stream?

Collection:

```
Java
List<Integer> list;
```

Stores data.

Stream:

```
Java
list.stream();
```

Processes data.

Q4. What is Optional?

Wrapper object used to avoid NullPointerException.

Q5. What is a Record?

```
Java
record Employee(String name, int age){}
```

Immutable data carrier.

Q6. What is a Sealed Class?

Restricts inheritance.

```
Java
sealed class Vehicle permits Car,Bike
```

Q7. What are Virtual Threads?

Lightweight JVM-managed threads introduced in Java 21.

Benefits:

- Less memory
- Massive concurrency
- Better scalability

Q8. Difference between Platform Thread and Virtual Thread?

Platform Thread	Virtual Thread
OS Thread	JVM Thread
Heavy	Lightweight
Limited	Millions possible
More memory	Less memory

Q9. What are Stream Intermediate and Terminal Operations?

Intermediate:

```
Java
filter()
map()
sorted()
distinct()
```

Terminal:

```
Java
collect()
forEach()
count()
findFirst()
```

Q10. Why Java 17 is popular?

Because it is an LTS release and includes:

- Records
- Sealed Classes
- Pattern Matching
- Better performance

Questions Frequently Asked to Spring Boot Developers

Since you're a Spring Boot developer, expect:

1. Explain Lambda Expressions.
2. Functional Interface vs Normal Interface.
3. Stream API lifecycle.
4. Optional usage.
5. Record vs POJO.
6. Sealed Class vs Final Class.
7. Virtual Threads in Java 21.
8. Pattern Matching in Java 17.
9. `map()` vs `flatMap()`.
10. Parallel Stream vs Stream.
11. `CompletableFuture`.
12. `ForkJoinPool`.
13. `ExecutorService`.
14. `Thread` vs `Runnable` vs `Callable`.
15. Java Memory Model.
16. Garbage Collection.
17. `HashMap` Internal Working.
18. `ConcurrentHashMap`.
19. `Comparable` vs `Comparator`.
20. `equals()` vs `hashCode()`.

For a **2-year Java/Spring Boot interview**, I'd prioritize:

1. Java 8 (Streams, Lambda, Optional) ★★★★★
2. Collections & `HashMap` ★★★★★
3. Multithreading & `ExecutorService` ★★★★★
4. Java 17 (Records, Sealed Classes) ★★★★★
5. Java 21 (Virtual Threads) ★★★
6. JVM, GC, Memory ★★★★★

These topics appear in the majority of Java backend interviews today.

Features of Java 8

Java 8 Features (Most Important for Interviews)

Java 8 is one of the most significant releases of Java. If you're interviewing for a Java/Spring Boot role, Java 8 features are asked very frequently.

1. Lambda Expressions

Allows you to write anonymous functions in a concise way.

Before Java 8

```
Java
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("Running");
    }
};
```

Java 8

```
Java
Runnable r = () -> System.out.println("Running");
```

Benefits

- Less boilerplate code
- Improved readability
- Enables functional programming

2. Functional Interfaces

An interface with exactly **one abstract method**.

Example

```
Java
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}
```

Common Built-in Functional Interfaces

Interface	Method
Predicate<T>	test()
Function<T,R>	apply()
Consumer<T>	accept()
Supplier<T>	get()

3. Stream API

Used for processing collections in a functional style.

Example

```
Java
List<Integer> nums = Arrays.asList(1,2,3,4,5);

nums.stream()
    .filter(n -> n % 2 == 0)
    .forEach(System.out::println);
```

Output:

```
2
4
```

Common Stream Operations

Intermediate Operations

```
Java
filter()
map()
sorted()
distinct()
limit()
skip()
```

Terminal Operations

```
Java
collect()
forEach()
count()
findFirst()
reduce()
```

4. Method References

Shorthand for lambda expressions.

Lambda

```
Java
list.forEach(x -> System.out.println(x));
```

Method Reference

Java

```
list.forEach(System.out::println);
```

Types

Java

```
ClassName::staticMethod
object::instanceMethod
ClassName::instanceMethod
ClassName::new
```

5. Optional Class

Helps avoid `NullPointerException`.

Example

Java

```
Optional<String> name =
    Optional.ofNullable(null);

System.out.println(name.orElse("Default"));
```

Output:

Default

Common Methods

Java

```
isPresent()
orElse()
orElseGet()
orElseThrow()
ifPresent()
```

6. New Date and Time API

Package:

Java

`java.time`

Classes

Class	Purpose
<code>LocalDate</code>	Date only
<code>LocalTime</code>	Time only
<code>LocalDateTime</code>	Date + Time
<code>Period</code>	Difference in dates

Class	Purpose
Duration	Difference in time

Example

```
Java
LocalDate today = LocalDate.now();

System.out.println(today);
```

7. Default Methods in Interfaces

Interfaces can have method implementations.

Example

```
Java
interface Vehicle {

    default void start() {
        System.out.println("Vehicle Started");
    }
}
```

8. Static Methods in Interfaces

```
Java
interface MathUtil {

    static int add(int a, int b) {
        return a + b;
    }
}
```

Usage:

```
Java
MathUtil.add(10,20);
```

9. forEach() Method

Simplified iteration.

Before Java 8

```
Java
for(String name : names){
    System.out.println(name);
}
```

Java 8

Java

```
names.forEach(System.out::println);
```

10. Nashorn JavaScript Engine

Allowed JavaScript execution inside Java.

Java

```
ScriptEngine engine =
    new ScriptEngineManager()
        .getEngineByName("nashorn");
```

⚠ Removed in later Java versions.

11. Parallel Streams

Uses multiple CPU cores.

Example

Java

```
list.parallelStream()
    .forEach(System.out::println);
```

Benefit

Faster processing of large datasets.

12. CompletableFuture

Improved asynchronous programming.

Example

Java

```
CompletableFuture<String> future =
    CompletableFuture.supplyAsync(
        () -> "Hello"
    );

System.out.println(future.get());
```

Most Asked Java 8 Interview Questions

1. What are the major features of Java 8?

Answer:

- Lambda Expressions
- Functional Interfaces
- Stream API
- Method References
- Optional
- New Date-Time API
- Default Methods
- CompletableFuture

2. What is a Functional Interface?

An interface with exactly one abstract method.

```
Java
@FunctionalInterface
interface Test {
    void run();
}
```

3. What is the difference between Collection and Stream?

Collection	Stream
Stores data	Processes data
Can be modified	Cannot modify source
Eager	Lazy

4. What is Lazy Evaluation in Streams?

Intermediate operations execute only when a terminal operation is called.

```
Java
list.stream()
    .filter(x -> x > 5)
    .count();
```

5. Difference between map() and flatMap()?

map()

One input → One output

Java

```
List<String> names =
    List.of("java", "spring");

names.stream()
    .map(String::toUpperCase);
```

flatMap()

Flattens nested structures.

Java

```
list.stream()
    .flatMap(List::stream);
```

6. Difference between findFirst() and findAny()? ---

- `findFirst()` → first element
 - `findAny()` → any element (useful in parallel streams)
-

7. What is Optional? ---

A container object that may or may not contain a value.

Used to reduce `NullPointerException`.

8. Difference between Predicate, Function, Consumer, Supplier? ---

Interface	Method	Returns
Predicate	<code>test()</code>	boolean
Function	<code>apply()</code>	value
Consumer	<code>accept()</code>	void
Supplier	<code>get()</code>	value

9. What is Method Reference? ---

Short form of lambda.

Java

```
System.out::println
```

10. What is Parallel Stream?

A stream that processes elements concurrently using the ForkJoin framework.

```
Java  
List.parallelStream()
```

For Your Java/Spring Boot Interviews

Focus heavily on:

1. Lambda Expressions
2. Functional Interfaces
3. Stream API (**filter**, **map**, **flatMap**, **reduce**, **collect**)
4. Optional
5. Method References
6. CompletableFuture
7. Default Methods
8. New Date-Time API

These are among the most commonly asked Java 8 topics in backend developer interviews.